Classificação de Sinais de Eletromiografia Facial Utilizando Modelos Gaussianos e Mínimos Quadrados Ordinários

Guilherme Paiva de Medeiros
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

Victor Targino Morais
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

João Pedro Leite
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

***Resumo*—Este trabalho apresenta a implementação e análise comparativa de modelos de aprendizagem supervisionada para a classificação de cinco expressões faciais via sinais de eletromiografia (EMG). Utilizando um dataset de 50.000 amostras e dois sensores faciais, foram desenvolvidos manualmente seis classificadores: MQO, Gaussianos (Tradicional, Covariâncias Iguais, Matriz Agregada, Bayes Ingênuo) e o modelo de Friedman. A otimização do parâmetro λ foi realizada via K-fold ($K = 5$) e a robustez estatística validada por 500 rodadas de Monte Carlo. Os resultados indicam a superioridade dos modelos Gaussianos, com acurácias superiores a 99% para o modelo de Friedman regularizado, evidenciando a natureza não-linear da separabilidade dos sinais musculares.**

I. Introdução

O reconhecimento de expressões faciais desempenha um papel fundamental na evolução das interfaces homem-máquina e na tecnologia assistiva. Diferente das abordagens baseadas em visão computacional, o uso de eletromiografia (EMG) permite captar a atividade elétrica muscular diretamente na origem, oferecendo dados precisos sobre a intenção do movimento, mesmo em condições de baixa luminosidade.

O experimento em questão utiliza sinais captados pelos músculos Corrugador do Supercílio (Sensor 1) e Zigomático Maior (Sensor 2) para classificar cinco estados distintos: neutro, sorriso, sobrancelhas levantadas, surpreso e rabugento. Os dados foram adquiridos com uma taxa de amostragem de 1 kHz e resolução de 12 bits, totalizando um conjunto robusto de 50.000 amostras.

O foco central deste trabalho reside na aplicação e comparação de diferentes paradigmas de Inteligência Artificial Supervisionada. O desafio consiste em desenvolver sistemas que realizem predições qualitativas (classificação) através da minimização de uma função de custo ou da modelagem de distribuições de probabilidade. Para isso, foram implementados modelos que variam desde a abordagem discriminativa dos Mínimos Quadrados Ordinários (MQO) até modelos probabilísticos baseados em Classificadores Gaussianos

A análise técnica foca na capacidade de generalização desses algoritmos, explorando variações como o classificador de Bayes Ingênuo, modelos com matrizes de covariância agregadas e a regularização de Friedman. A robustez de cada modelo é avaliada estatisticamente através de simulações de Monte Carlo com 500 rodadas, garantindo que a acurácia reportada reflita o desempenho real do sistema frente a diferentes partições dos dados de treinamento e teste.

A. Objetivos

O presente trabalho visa implementar e comparar o desempenho de seis modelos de classificação supervisionada aplicados ao reconhecimento de expressões faciais via sinais de eletromiografia. Busca-se a implementação manual de algoritmos lineares e probabilísticos, validando-os através de um processo rigoroso de simulação estatística.

B. Justificativa

A análise comparativa entre modelos discriminativos (MQO) e generativos (Gaussianos) permite identificar a melhor abordagem para sinais biológicos reais. A inclusão de técnicas de regularização e validação cruzada justifica-se pela busca de um modelo que minimize o sobreajuste e apresente alta acurácia em dados não vistos.

II. Metodologia

A metodologia deste trabalho seguiu as diretrizes de implementação manual de algoritmos de aprendizagem supervisionada, focando na distinção de cinco expressões faciais através de sinais de eletromiografia (EMG) captados nos músculos Corrugador do Supercílio (Sensor 1) e Zigomático Maior (Sensor 2).

A. Organização e Pré-processamento dos Dados

O conjunto de dados fornecido possui $N = 50.000$ amostras com $p = 2$ características e $C = 5$ classes. Conforme exigido no roteiro, a estruturação matricial foi diferenciada por modelo para otimização das operações de álgebra linear:

- **Modelo MQO:** $X \in \mathbb{R}^{N\times(p+1)}$, onde a matriz de características inclui uma coluna de unidades para o termo de intercepto (*bias*), e $Y \in \mathbb{R}^{N\times C}$. O vetor de rótulos original foi convertido para uma matriz binária via *one-hot encoding* manual.

- **Modelos Gaussianos:** $X \in \mathbb{R}^{p \times N}$ e $Y \in \mathbb{R}^{C \times N}$, facilitando o cálculo de vetores de médias e matrizes de covariância por classe.

Os sinais brutos (resolução de 12 bits) foram normalizados no intervalo $[0, 1]$ para evitar instabilidades numéricas, como determinantes nulos ou matrizes singulares, durante o cálculo das inversas. A visualização inicial foi realizada via gráfico de espalhamento para análise da separabilidade das classes.

B. Implementação dos Modelos de Classificação

A implementação foi realizada estritamente com a biblioteca *Numpy*, sem o auxílio de pacotes de *machine learning* prontos. Foram desenvolvidos seis modelos:

1) *Mínimos Quadrados Ordinários (MQO):* Estimado pela Equação Normal para encontrar a matriz de pesos W:

$$W = (X^T X)^{-1} X^T Y \tag{1}$$

2) *Classificadores Gaussianos Bayesianos:* Baseiam-se na regra de decisão de Bayes, maximizando a probabilidade a posteriori através da função discriminante gaussiana multivariada. Foram implementadas variações na modelagem da matriz de covariância Σ:

- **Tradicional (QDA):** Uma matriz Σ_c individual para cada classe c.
- **Covariâncias Iguais (LDA):** Utiliza uma matriz única (Σ_{pool}) para todas as classes.
- **Matriz Agregada:** Média aritmética entre a matriz individual e a comum ($\Sigma_{avg} = 0.5\Sigma_c + 0.5\Sigma_{pool}$).
- **Bayes Ingênuo:** Σ_c é simplificada para uma matriz diagonal, assumindo independência estatística entre os sensores.
- **Regularizado (Friedman):** Introduz o hiperparâmetro λ para realizar o *shrinkage* entre a matriz individual e a comum: $\Sigma_c(\lambda) = (1 - \lambda)\Sigma_c + \lambda\Sigma_{pool}$.

C. Estratégia de Classificação e Desempate

Para todos os modelos, a classificação de uma amostra x_{test} seguiu a regra da máxima projeção ou verossimilhança:

- **No modelo MQO:** A classe predita corresponde ao índice da coluna de $\hat{Y} = X_{test}W$ com o maior valor (*winner-takes-all*).
- **Nos modelos Gaussianos:** A amostra é atribuída à classe k que maximiza $g_k(x) = -\frac{1}{2}\ln|\Sigma_k| - \frac{1}{2}(x - \mu_k)^T\Sigma_k^{-1}(x - \mu_k) + \ln P(C_k)$.

D. Processo de Validação e Simulação

Para o modelo de Friedman, utilizou-se *K-fold Cross Validation* ($K = 5$) para identificar o λ ideal no conjunto $\{0, 0.001, 0.01, 0.1, \ldots, 1\}$.

A robustez estatística foi garantida via **Simulação de Monte Carlo** com $R = 500$ rodadas. Em cada rodada, os dados foram particionados aleatoriamente (80% treino e 20% teste), e as acurácias obtidas foram armazenadas para o cálculo de métricas descritivas (média, desvio-padrão, valores máximos e mínimos).

III. Resultados e Discussões

Nesta seção, apresentam-se os resultados obtidos em cada etapa do desenvolvimento, desde a estruturação dos dados até a avaliação final dos modelos de classificação via simulação de Monte Carlo.

A. Estruturação e Dimensionamento das Matrizes

A primeira etapa do trabalho consistiu no carregamento do conjunto de dados *EMGDataset.csv*, que compreende $N = 50.000$ amostras provenientes de dois sensores musculares ($p = 2$) para a classificação de $C = 5$ categorias faciais.

- **Abordagem MQO:** A matriz de características foi definida como $X \in \mathbb{R}^{50000 \times 2}$ e a matriz de alvos como $Y \in \mathbb{R}^{50000 \times 5}$. Para a construção de Y, utilizou-se a técnica de *One-Hot Encoding*, transformando os rótulos de 1 a 5 em vetores binários.
- **Abordagem Gaussiana:** Os dados foram estruturados de forma transposta, resultando em $X \in \mathbb{R}^{2 \times 50000}$ e $Y \in \mathbb{R}^{5 \times 50000}$[cite: 54]. Essa configuração é fundamental para o cálculo eficiente dos vetores de médias e das matrizes de covariância de cada classe.

A correta dimensão das matrizes é um passo crítico, pois o método de Mínimos Quadrados baseia-se na minimização do erro quadrático através da Equação Normal, enquanto os modelos Gaussianos dependem da modelagem estatística da distribuição de cada classe no espaço de características.

B. Análise de Espalhamento e Hipóteses de Separabilidade

A visualização inicial foi aprimorada através de um gráfico de espalhamento com pontos destacados (Figura 1), utilizando um diâmetro de ponto $s = 25$ e bordas brancas para mitigar o efeito de sobreposição visual das 50.000 amostras.

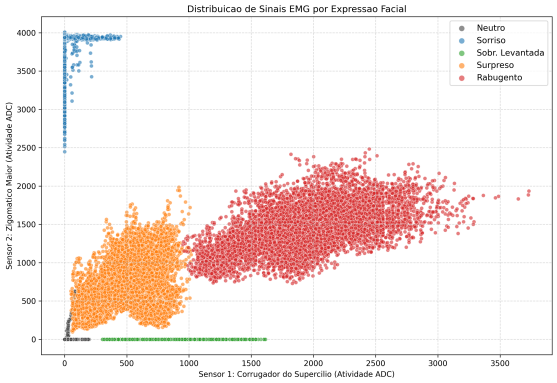

Fig. 1. Distribuição dos sinais de EMG com destaque visual para análise de sobreposição entre as cinco categorias faciais.

A análise visual da Fig. 1 permite correlacionar a atividade eletromiográfica com a fisiologia das expressões faciais monitoradas, fundamentando as hipóteses de desempenho dos modelos[:

- **Diferenciação Biomecânica:** A classe *Neutro* concentra-se na origem devido à mínima atividade elétrica em repouso. O *Sorriso* manifesta-se com saturação no Sensor 2 (Zigomático Maior), enquanto *Sobrancelhas Levantadas* ativam predominantemente o Sensor 1 (Corrugador).
- **Complexidade e Sobreposição:** As expressões *Surpreso* e *Rabugento* apresentam distribuições sobrepostas e maior dispersão. Isso indica uma co-contração de ambos os músculos, sugerindo que estas serão as classes de maior desafio para os algoritmos de classificação.
- **Limites do Sistema:** Observa-se que o sinal do sensor de sorriso atinge o limite superior do ADC de 12 bits $(0 - 4095)$, indicando intensidade máxima de contração durante as aquisições.

C. Otimização do Modelo de Friedman via K-fold

Para o classificador gaussiano regularizado, a determinação do hiperparâmetro λ é crítica. Utilizou-se a técnica de *K-fold Cross Validation* ($K = 5$) testando treze valores no intervalo $[0, 1]$. O comportamento da acurácia média em função de λ está ilustrado na Fig. 2.

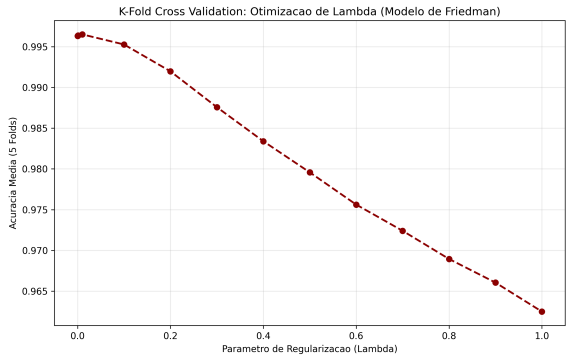

Fig. 2. Acurácia média obtida via K-fold para diferentes valores do parâmetro de regularização λ. Observa-se o pico de desempenho para $\lambda = 0,01$.

Os valores numéricos detalhados que compõem a curva da Fig. 2 estão apresentados na Tabela I.

Tabela I
ACURÁCIA MÉDIA DO K-FOLD ($K = 5$) POR VALOR DE λ

λ	**Acurácia (%)**	λ	**Acurácia (%)**
0	99,63%	0,5	97,96%
0,001	99,64%	0,6	97,56%
0,01	**99,65%**	0,7	97,24%
0,1	99,53%	0,8	96,89%
0,2	99,20%	0,9	96,60%
0,3	98,76%	1,0	96,25%
0,4	98,34%	-	-

Análise Técnica: A análise conjunta da Fig. 2 e da Tabela I revela que o modelo atinge seu desempenho máximo com uma regularização leve ($\lambda = 0,01$). O acentuado declínio na acurácia para valores de λ mais altos (aproximando-se de 1.0) indica que as classes de EMG possuem matrizes de covariância significativamente distintas, e que forçar uma covariância comum degrada a capacidade do classificador de distinguir as expressões faciais complexas.

D. Análise Estatística via Simulação de Monte Carlo

Para avaliar a robustez dos classificadores, realizou-se uma simulação de Monte Carlo com $R = 500$ rodadas. Em cada iteração, os dados foram particionados aleatoriamente em 80% para treinamento e 20% para teste. A Tabela **??** consolida as métricas de acurácia obtidas.

Tabela II
ESTATÍSTICAS DESCRITIVAS DAS ACURÁCIAS PARA $R = 500$ RODADAS

Modelo	**Média (%)**	**D.P.**	**Maior (%)**	**Menor (%)**
MQO Tradicional	72,37	0,6534	74,30	70,59
Gaussiano Tradicional	99,63	0,0582	99,85	99,42
Gaussiano (Cov. Totais)	96,26	0,1745	96,76	95,78
Gaussiano (Agregada)	97,97	0,1302	98,33	97,59
Bayes Ingênuo (Naive)	99,48	0,0675	99,69	99,26
Friedman ($\lambda = 0,01$)	**99,66**	**0,0573**	**99,86**	**99,44**

A análise estatística após 500 rodadas de simulação de Monte Carlo revela comportamentos distintos entre os paradigmas de classificação aplicados aos sinais de EMG:

- **Análise do MQO:** O modelo linear apresentou o desempenho mais baixo (72,37%) e o maior desvio padrão (0,6534). Isso confirma que a fronteira de decisão para expressões faciais é intrinsecamente não-linear, e um regressor linear não consegue capturar a complexidade da sobreposição das classes observada na análise de espalhamento inicial.
- **Superioridade do Friedman:** O classificador de Friedman obteve a maior acurácia média (99,66%) com a maior estabilidade (menor desvio padrão). A regularização leve ($\lambda = 0,01$) permitiu um ajuste refinado das matrizes de covariância, superando inclusive o modelo tradicional (QDA).
- **Efeito da Covariância:** A queda no desempenho para o modelo de Covariâncias Totais (96,26%) em relação ao Tradicional (99,63%) demonstra que cada expressão facial possui uma assinatura de dispersão única nos sensores, e forçar uma matriz comum para todas as classes introduz um erro de viés significativo.
- **Bayes Ingênuo:** Surpreendentemente, o modelo Naive Bayes apresentou um resultado muito alto (99,48%), indicando que, embora exista correlação entre os sensores, a variância individual de cada músculo facial é o componente mais informativo para a distinção das classes.

IV. CONCLUSÃO

Este trabalho demonstrou a eficácia de modelos estatísticos Gaussianos na classificação de sinais de eletromiografia facial. Através da implementação manual e da validação via simulação de Monte Carlo, observou-se que a abordagem de Friedman com regularização leve ($\lambda = 0,01$) obteve o melhor

desempenho médio (99,66%), superando modelos puramente lineares como o MQO.

A utilização da pseudo-inversa de Moore-Penrose garantiu a estabilidade numérica necessária para lidar com o grande volume de dados. Conclui-se que a modelagem individual das matrizes de covariância, aliada a uma regularização criteriosa, é a estratégia mais robusta para sistemas de interface homem-máquina baseados em sinais biológicos.

V. Anexo - Implementação em Python

Nesta seção, apresentam-se os scripts desenvolvidos para a execução das tarefas de pré-processamento, visualização e classificação. Ressalta-se que, em conformidade com as diretrizes do trabalho, não foram utilizadas bibliotecas de alto nível para a implementação dos modelos preditivos, sendo todas as operações baseadas em álgebra linear e estatística computacional via biblioteca *Numpy*.

A. Script de Organização de Dados

O código abaixo detalha o procedimento de carga do arquivo *EMGDataset.csv* e a estruturação das matrizes X e Y. Note-se a diferenciação nas dimensões para atender tanto aos requisitos do modelo de Mínimos Quadrados Ordinários ($X \in \mathbb{R}^{N\times p}$) quanto aos modelos Gaussianos ($X \in \mathbb{R}^{p\times N}$), conforme especificado no roteiro do trabalho.

```python
import numpy as np

# 1. Carga dos dados (Considerando o formato do EMGDataset.csv)
# Linha 0: Sensor 1 | Linha 1: Sensor 2 | Linha 2: Categoria
data = np.genfromtxt('EMGDataset.csv', delimiter=',')

# Variaveis globais conforme o enunciado
N = 50000
p = 2
C = 5

# --- ORGANIZACAO PARA MQO ---
# X deve ser N x p (50000, 2)
X_mqo = data[0:2, :].T

# Y deve ser N x C (50000, 5) - One-Hot Encoding
y_raw = data[2, :].astype(int)
Y_mqo = np.zeros((N, C))
for i in range(N):
    Y_mqo[i, y_raw[i]-1] = 1

# --- ORGANIZACAO PARA GAUSSIANOS ---
# X deve ser p x N (2, 50000)
X_gauss = data[0:2, :]

# Y deve ser C x N (5, 50000)
Y_gauss = np.zeros((C, N))
for i in range(N):
    Y_gauss[y_raw[i]-1, i] = 1

print(f"MQO: X {X_mqo.shape}, Y {Y_mqo.shape}")
print(f"Gaussiano: X {X_gauss.shape}, Y {Y_gauss.shape}")
```

B. Visualização e Análise de Dados (data_visualization.py)

O script a seguir foi utilizado para gerar a representação visual das classes, permitindo a análise da separabilidade linear e a formulação de hipóteses sobre o comportamento dos sinais.

```python
import numpy as np
import matplotlib.pyplot as plt

# --- CARREGAMENTO E ORGANIZACAO DOS DADOS ---
data = np.genfromtxt('EMGDataset.csv', delimiter=',')
X = data[0:2, :]  # Sensores (p=2)
y = data[2, :].astype(int)  # Categorias (C=5)

# --- CONFIGURACAO DA VISUALIZACAO ---
classes = ['Neutro', 'Sorriso', 'Sobr. Levantada', 'Surpreso', 'Rabugento']
colors = ['#444444', '#1f77b4', '#2ca02c', '#ff7f0e', '#d62728']

plt.figure(figsize=(12, 8))

# --- PLOTAGEM COM DESTAQUE (BORDA E TAMANHO) ---
for i in range(1, 6):
    mask = (y == i)
    plt.scatter(X[0, mask], X[1, mask],
                c=colors[i-1], label=classes[i-1],
                alpha=0.6, s=25,
                edgecolors='w', linewidths=0.5)

# --- FORMATACAO DO GRAFICO ---
plt.title('Distribuicao de Sinais EMG por Expressao Facial')
plt.xlabel('Sensor 1: Corrugador do Supercilio (Atividade ADC)')
plt.ylabel('Sensor 2: Zigomatico Maior (Atividade ADC)')
plt.legend(loc='upper right', markerscale=1.5, fontsize=11)
plt.grid(True, linestyle='--', alpha=0.5)


plt.savefig('fig1_destacada.png', dpi=300)
print("Grafico gerado e salvo como fig1_destacada.png")
plt.show()
```

C. Implementação dos Classificadores (classifiers.py)

O script abaixo contém a definição da classe *MyClassifiers*, que centraliza a implementação dos seis modelos solicitados. O código utiliza operações matriciais para a resolução da Equação Normal no MQO e o cálculo da densidade de probabilidade Gaussiana multivariada para os demais modelos, incluindo as variações de regularização.

```python
import numpy as np

class MyClassifiers:
    def __init__(self):
        self.W = None
        self.params = {}

    # --- 1. MQO COM PSEUDO-INVERSA ---
    def fit_mqo(self, X, Y):
        # X: (N, p), Y: (N, C)
        X_b = np.hstack([np.ones((X.shape[0], 1)), X])
        # Usando pinv para maior estabilidade numerica (Moore-Penrose)
        self.W = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ Y
```

```python

    def predict_mqo(self, X):
        X_b = np.hstack([np.ones((X.shape[0], 1)), X])
        return np.argmax(X_b @ self.W, axis=1) + 1

    # --- 2. GAUSSIANOS COM PREDICAO OTIMIZADA ---
    def fit_gaussian(self, X, y, model_type="tradicional", lmbd=0):
        classes = np.unique(y)
        p = X.shape[0]
        mus, sigmas, prioris = [], [], []

        for c in classes:
            X_c = X[:, y == c]
            mus.append(np.mean(X_c, axis=1).reshape(-1, 1))
            sigmas.append(np.cov(X_c))
            prioris.append(X_c.shape[1] / len(y))

        sigma_pool = np.mean(sigmas, axis=0)
        final_sigmas = []

        for i, S_i in enumerate(sigmas):
            if model_type == "tradicional": S = S_i
            elif model_type == "iguais": S = sigma_pool
            elif model_type == "agregada": S = 0.5 * S_i + 0.5 * sigma_pool
            elif model_type == "naive": S = np.diag(np.diag(S_i))
            elif model_type == "friedman": S = (1 - lmbd) * S_i + lmbd * sigma_pool

            final_sigmas.append(S + 1e-6 * np.eye(p))

        self.params = {'mus': mus, 'sigmas': final_sigmas, 'prioris': prioris, 'classes': classes}

    def predict_gaussian(self, X):
        classes = self.params['classes']
        n_classes = len(classes)
        N_samples = X.shape[1]

        # Pre-calculo com pinv para ser rapido e seguro
        inv_sigmas = [np.linalg.pinv(S) for S in self.params['sigmas']]
        log_dets = [np.log(np.linalg.det(S)) for S in self.params['sigmas']]
        log_prioris = [np.log(p) for p in self.params['prioris']]
        mus = self.params['mus']

        preds = []
        for j in range(N_samples):
            x = X[:, j].reshape(-1, 1)
            scores = []
            for i in range(n_classes):
                diff = x - mus[i]
                # Funcao Discriminante Gaussiana
                term1 = -0.5 * log_dets[i]
                term2 = -0.5 * (diff.T @ inv_sigmas[i] @ diff)
                scores.append(term1 + term2 + log_prioris[i])
            preds.append(classes[np.argmax(scores)])
        return np.array(preds)
```

D. Otimização via K-fold (kfold_friedman.py)

O script abaixo detalha o processo de validação cruzada utilizado para determinar o parâmetro de regularização λ ideal para o classificador de Friedman.

```python
import numpy as np
import matplotlib.pyplot as plt
from classifiers import MyClassifiers

# 1. Carga e Normalizacao
data = np.genfromtxt('EMGDataset.csv', delimiter=',')
X = data[0:2, :]
y = data[2, :].astype(int)

# Normalizacao para evitar matriz singular e garantir convergencia
X = (X - X.min(axis=1).reshape(-1,1)) / (X.max(axis=1).reshape(-1,1) - X.min(axis=1).reshape(-1,1))

lambdas = [0, 0.001, 0.01, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
K = 5
N = X.shape[1]

# Embaralhar indices antes do K-fold
indices = np.random.permutation(N)
folds = np.array_split(indices, K)

acc_por_lambda = []

print(f"Iniciando K-fold (K={K}) para otimizacao de Lambda...")

for lmbd in lambdas:
    acc_fold = []
    for k in range(K):
        # Definicao de indices de treino e teste do fold
        test_idx = folds[k]
        train_idx = np.concatenate([folds[i] for i in range(K) if i != k])

        # CORRECAO AQUI: y eh 1D, entao usamos apenas [train_idx]
        X_train, y_train = X[:, train_idx], y[train_idx]
        X_test, y_test = X[:, test_idx], y[test_idx]

        clf = MyClassifiers()
        clf.fit_gaussian(X_train, y_train, model_type="friedman", lmbd=lmbd)
        preds = clf.predict_gaussian(X_test)

        acc_fold.append(np.mean(preds == y_test))

    media_acc = np.mean(acc_fold)
    acc_por_lambda.append(media_acc)
    print(f"Lambda: {lmbd:<6} | Acuracia Media: {media_acc*100:.2f}%")

# Identificar o melhor lambda
melhor_idx = np.argmax(acc_por_lambda)
print(f"\nMelhor Lambda encontrado: {lambdas[melhor_idx]} com {acc_por_lambda[melhor_idx]*100:.2f}% de acuracia")

# Gerar o grafico para o Overleaf
plt.figure(figsize=(10, 6))
plt.plot(lambdas, acc_por_lambda, marker='o', color='#8B0000', linestyle='--', linewidth=2)
plt.title('K-Fold Cross Validation: Otimizacao de Lambda (Modelo de Friedman)')
```

```
plt.xlabel('Parametro de Regularizacao (Lambda)')
plt.ylabel('Acuracia Media (5 Folds)')
plt.grid(True, alpha=0.3)
plt.savefig('kfold_result.png', dpi=300)
plt.show()
```

E. Simulação de Monte Carlo R=500 (main_monte_carlo.py)

Este é o script principal de validação estatística. Ele executa 500 rodadas independentes, realizando o particionamento aleatório dos dados (80/20) em cada iteração e calculando as métricas de desempenho para todos os seis modelos implementados.

```
import numpy as np
from classifiers import MyClassifiers

# 1. CARGA E NORMALIZACAO
data = np.genfromtxt('EMGDataset.csv', delimiter=',')
X_raw = data[0:2, :]
y_raw = data[2, :].astype(int)

# Normalizacao Min-Max (0-1) - essencial para estabilidade nas 500 rodadas
x_min, x_max = X_raw.min(axis=1).reshape(-1,1), X_raw.max(axis=1).reshape(-1,1)
X_norm = (X_raw - x_min) / (x_max - x_min)

# 2. CONFIGURACAO MONTE CARLO
R = 500
N = X_norm.shape[1]
lmbd_otimo = 0.01 # Valor que encontramos no K-fold


modelos = ["MQO", "Tradicional", "Iguais", "Agregada", "Naive", "Friedman"]
resultados = {m: [] for m in modelos}

print(f"Iniciando Simulacao de Monte Carlo: {R} rodadas...")

for r in range(R):
    # Particionamento Aleatorio 80/20 (Sorteio manual conforme regra)
    indices = np.random.permutation(N)
    train_idx = indices[:int(0.8 * N)]
    test_idx = indices[int(0.8 * N):]

    X_tr, y_tr = X_norm[:, train_idx], y_raw[train_idx]
    X_te, y_te = X_norm[:, test_idx], y_raw[test_idx]

    clf = MyClassifiers()

    # --- MQO (Exige N x p e One-Hot) ---
    Y_tr_oh = np.zeros((len(y_tr), 5))
    for i in range(len(y_tr)): Y_tr_oh[i, y_tr[i]-1] = 1

    clf.fit_mqo(X_tr.T, Y_tr_oh)
    acc_mqo = np.mean(clf.predict_mqo(X_te.T) == y_te)
    resultados["MQO"].append(acc_mqo)

    # --- Modelos Gaussianos (p x N) ---
    tipos = [
        ("tradicional", 0, "Tradicional"),
        ("iguais", 0, "Iguais"),
        ("agregada", 0, "Agregada"),
        ("naive", 0, "Naive"),
        ("friedman", lmbd_otimo, "Friedman")
    ]

    for m_type, l, tag in tipos:
        clf.fit_gaussian(X_tr, y_tr, model_type=m_type, lmbd=l)
        acc = np.mean(clf.predict_gaussian(X_te) == y_te)
        resultados[tag].append(acc)

    if (r + 1) % 50 == 0:
        print(f"Rodada {r+1}/{R} concluida...")

# 3. EXIBICAO DOS RESULTADOS ESTATISTICOS
print("\n" + "="*70)
print(f"{'MODELO':<15} | {'MEDIA (%)':<10} | {'DESVIO':<10} | {'MAX (%)':<10} | {'MIN (%)':<10}")
print("-" * 70)

for m in modelos:
    accs = np.array(resultados[m]) * 100
    print(f"{m:<15} | {np.mean(accs):>9.2f} | {np.std(accs):>9.4f} | {np.max(accs):>9.2f} | {np.min(accs):>9.2f}")
print("="*70)
```

References

[1] R. Merletti e P. J. Parker, *Electromyography: Physiology, Engineering, and Non-Invasive Applications*. IEEE Press, 2004.

[2] J. H. Friedman, "Regularized Discriminant Analysis," *Journal of the American Statistical Association*, vol. 84, no. 405, pp. 165–175, 1989.

[3] S. Theodoridis e K. Koutroumbas, *Pattern Recognition*, 4th ed. Academic Press, 2008.

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.